

NIVEL 1

Exercici 1:

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Para este ejercicio, creamos las tablas company y transaction que viene con relaciones definidas de FOREIGN KEY en company (id) y credit card (id).

```
2 • CREATE DATABASE IF NOT EXISTS transactions;
3 • USE transactions;
4
5 -- Creamos la tabla company
6 • CREATE TABLE IF NOT EXISTS company (
7   id VARCHAR(15) PRIMARY KEY,
8   company_name VARCHAR(255),
9   phone VARCHAR(15),
10  email VARCHAR(100),
11  country VARCHAR(100),
12  website VARCHAR(255)
13 );
14
15 -- Creamos la tabla transaction
16 • CREATE TABLE IF NOT EXISTS transaction (
17   id VARCHAR(255) PRIMARY KEY,
18   credit_card_id VARCHAR(15) REFERENCES credit_card(id),
19   company_id VARCHAR(20),
20   user_id INT REFERENCES user(id),
21   lat FLOAT,
22   longitude FLOAT,
23   timestamp TIMESTAMP,
24   amount DECIMAL(10, 2),
25   declined BOOLEAN,
26   FOREIGN KEY (company_id) REFERENCES company(id)
27 );
--
```

Output

Action Output

#	Time	Action
1	22:56:12	CREATE DATABASE IF NOT EXISTS transactions
2	22:56:12	USE transactions
3	22:56:12	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_n
4	22:56:12	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_c

Ejecutamos el archivo dades_introduir que contiene los datos de las tablas transactions y companies, ejecutamos y hacemos el insert en las tablas.

```
10002 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('1771793A-0940-44F0-8095-07473116A002', 'C5-6688', '0-2484', '2107', '52.46315388327127', '4.3962239104958895', '2010-05-07 01:55:00', '126.36', '0')
10003 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('7779A2D0-0310-4017-8F0D-078F05A00308', 'C5-6094', '0-2102', '2103', '51.8386872708682', '5.603186661647654', '2017-03-30 13:53:10', '108.58', '0')
10070 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('7C801940-3288-478F-8238-2C946C4F989', 'C5-8435', '0-2438', '3054', '46.3818958853571', '1.544886644258322', '2018-08-15 13:53:56', '655.32', '0')
10071 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('A6588598-3427-486A-8348-8E6A732AA593', 'C5-8288', '0-2202', '3099', '46.45494451227346', '-3.794887543882871', '2022-03-09 05:39:44', '114.77', '0')
10072 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('7208007A-227A-437E-9806-F30828411824', 'C5-4998', '0-2366', '417', '39.28831896112337', '-6.10355382117661', '2021-08-28 13:27:57', '508.08', '0')
```

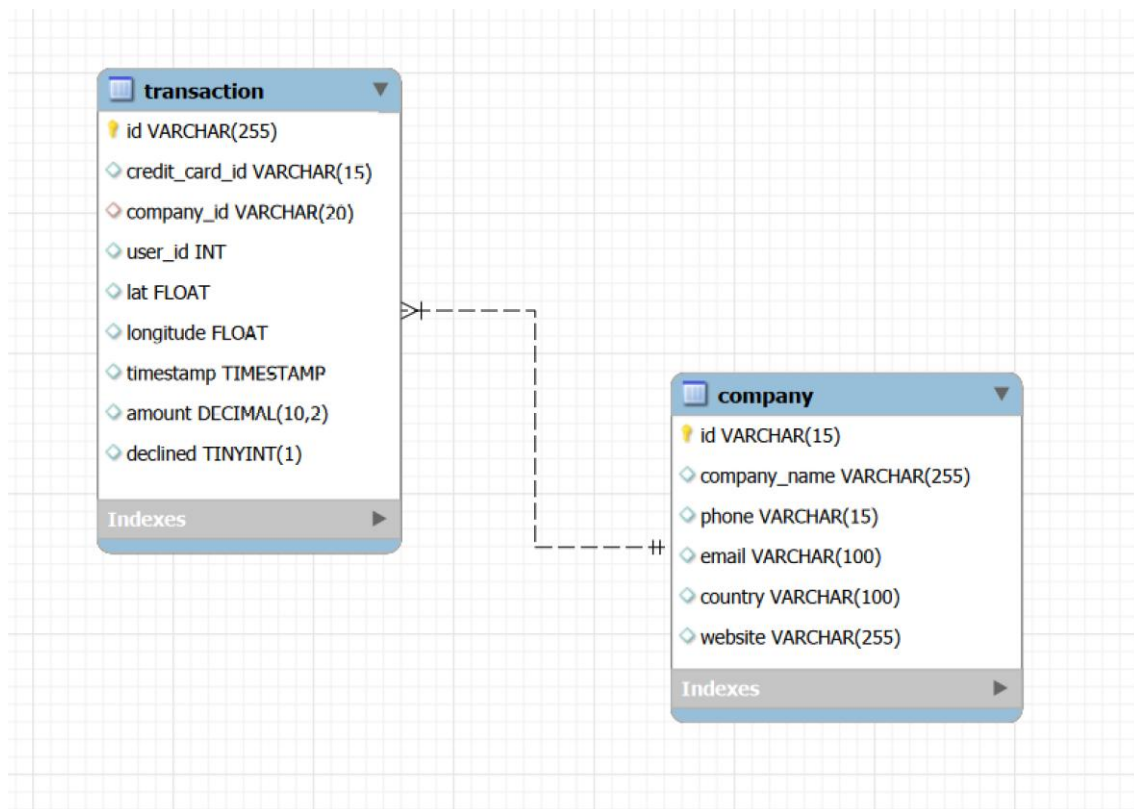
Output

Action Output

#	Time	Action	Message
5101	23:11:09	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('ACAC1E1E475-4E3A-903A83E7C40808', 'C5-6302', '0-2089', '1701', '52.0782644403605', '4.30026202052022', '2021-06-02 12:41:11', '108.2', '0')	Transaction affected
5102	23:11:09	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('5C7086D39F46707088F5488D1019A2', 'C5-7071', '0-2462', '3298', '59.33891888432844', '10.87686466540204', '2022-05-31 18:53:27', '714', '0')	Transaction affected
5103	23:11:09	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('A27086238024E318158-20183C24F0A', 'C5-5181', '0-2442', '930', '51.5248721019442', '10.2090846118225', '2017-12-26 14:04:02', '199.62', '0')	Transaction affected

Se observa una base de datos relacional llamada: “Transactions”, está compuesta por 2 tablas.

- La tabla principal es “transaction”: se puede apreciar que como clave primaria esta “transaction.id”. En esta tabla es donde ocurrirán los hechos ya que permiten medir, por fecha (timestamp), importe (amount), posición geográfica (longitude,lat), id de usuario(user_id), id de tarjeta de crédito (credit_card_id), el campo (declined) que indica si la transacción ha sido rechazada y también contiene company_id, que permite asociar la transacción con una empresa en “company”.
- La tabla “company”, contiene la información descriptiva de las empresas. Su clave primaria “company.id” y el resto de campos, nombre (company_name), teléfono(phone), mail(email), país(country) y su web(website).
- La relación que se observa es de uno a muchos (1:N) y se interpreta como: una empresa puede tener muchas transacciones y muchas transacciones están asociadas a una empresa mediante (transaction.company_id) y (company.id).



Exercici 2:

2.1 Listado de países que generan ventas:

```
38 # 2.1 Llistat dels països que estan generant vendes.
39 • SELECT DISTINCT(co.country) AS païses_vendas
40 FROM company AS co
41 JOIN transaction AS tr ON tr.company_id=co.id
42 WHERE tr.declined = 0 AND tr.amount >= 0;
43
```

Result Grid

païses_vendas
Netherlands
Sweden
Ireland
United States
Belgium
Canada
Germany
Norway

Result 3 x

Output

Action Output

#	Time	Action	Message
✓ 1	18:49:03	SELECT DISTINCT(co.country) AS païses_vendas FROM company AS ...	16 row(s) returned

2.2 ¿Desde cuantos países se generan las ventas?

```
45 # 2.2 Des de quants països es generen les vendes.
46 • SELECT COUNT(DISTINCT co.country) AS nro_païses_vendas
47 FROM company AS co
48 INNER JOIN transaction AS tr ON tr.company_id=co.id
49 WHERE tr.declined = 0 AND tr.amount >=0;
50
```

Result Grid

nro_païses_vendas
15

Output

Action Output

#	Time	Action	Message
✓ 1	18:51:56	SELECT COUNT(DISTINCT co.country) AS nro_païses_vendas FROM c...	1 row(s) returned

2.3 Identifica la compañía con la media más grande de ventas.

```
52 # 2.3 Identifica la companyia amb la mitjana més gran de vendes.
53 • SELECT co.company_name AS empresa, ROUND(AVG(tr.amount),2) as media_ventas
54 FROM company AS co
55 JOIN transaction as tr ON tr.company_id=co.id
56 WHERE tr.declined = 0
57 GROUP BY co.id
58 ORDER BY media_ventas DESC
59 LIMIT 1;
60
61
```

empresa	media_ventas
Ac Fermentum Incorporated	284.91

Result 81 x

Output

Action Output

#	Time	Action	Message
1	14:45:06	SELECT co.company_name AS empresa, ROUND(AVG(tr.amount),2) as media_ventas FROM company A...	1 row(s) returned

Exercici 3:

Utilitzant només subconsultes (sense utilitzar JOIN):

3.1 Muestra todas las transacciones realizadas por empresas de Alemania:

```
67 # 3.1 Mostra totes les transaccions realitzades per empreses d'Alemanya.
68 • SELECT *
69 FROM transaction
70 WHERE EXISTS (
71     SELECT id
72     FROM company
73     WHERE country='Germany'
74     AND declined=0
75     AND id=company_id # y relacionamos los id de company
76 );
77
```

id	credit_card_id	company_id	user_id	lat	longitude	time
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	202
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	202
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	202
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	202
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	202
00687139-48B2-4FFA-8E73-B20376F04AB4	CcS-4870	b-2222	289	51.1966	10.4669	201

transaction 8 x

Output

Action Output

#	Time	Action	Message
1	19:00:23	SELECT * FROM transaction WHERE EXISTS (SELECT id FROM com...	13269 row(s) returned

3.2 Lista de las empresas que han realizado transacciones por un monto superior a la media de todas las transacciones:

```
79 # 3.2 Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.
80 • SELECT
81 (SELECT co.company_name
82 FROM company AS co
83 WHERE co.id = tr.company_id) AS company_name,
84 ROUND(AVG(tr.amount), 2) AS media_ventas
85 FROM transaction AS tr
86 WHERE tr.declined = 0
87 AND tr.amount > (
88 SELECT AVG(t.amount)
89 FROM transaction AS t
90 WHERE t.declined = 0
91 )
92 GROUP BY tr.company_id
93 ORDER BY media_ventas DESC;
```

company_name	media_ventas
Ac Fermentum Incorporated	417.13
Sed LLC	408.99
Mattis Foundation	407.87
At Associates	401.93
Eget Ipsum Ltd	401.00

Result 14 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:09:15	SELECT (SELECT co.company_name FROM company AS co	100 row(s) returned	0.266 sec / 0.0

3.3 Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas:

```
100 # 3.3 Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.
101 • SELECT company_name AS empresas
102 FROM company AS co
103 WHERE NOT EXISTS (
104 SELECT tr.company_id
105 FROM transaction AS tr);
106
```

empresas

company 18 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:13:03	SELECT company_name AS empresas FROM company AS co WHERE...	0 row(s) returned	0.000 sec / 0.00

Exercici 4:

Primero creamos la tabla credit card, nos fijamos del INSERT los campos para replicarlos en la tabla.

Nos encontramos que la fecha viene por defecto en formato 'mes/dia/año' con lo cual, para leer fechas en sql no sería correcto. Crearemos como VARCHAR y luego lo vamos a transformar con la función STR_TO_DATE y aplicamos LIMIT con un número mucho más alto de los datos del INSERT para evitar restricciones de mysql.

Luego solo nos queda modificar la columna por DATE.

```
116 • CREATE TABLE IF NOT EXISTS credit_card (  
117     id VARCHAR(15) PRIMARY KEY,  
118     iban VARCHAR(255),  
119     pan VARCHAR(255),  
120     pin VARCHAR(10),  
121     cvv VARCHAR(10),  
122     expiring_date VARCHAR(50)  
123 );  
124  
125 #convertir de texto varchar a formato fecha  
126 • UPDATE credit_card  
127     SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')  
128     LIMIT 999999;  
129 #convertir a date  
130 • ALTER TABLE credit_card MODIFY COLUMN expiring_date DATETIME;
```

Output

#	Time	Action	Message
✓ 1	20:47:22	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIM...	0 row(s) affected
✓ 2	20:47:49	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_dat...	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0
✓ 3	20:47:53	ALTER TABLE credit_card MODIFY COLUMN expiring_date DATETIME	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Procedemos a realizar el INSERT en la tabla de credit_card:

```
4988 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9568', 'XX386536547822381431747659', '2296680186:  
4989 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9569', 'XX385272728964047897361563', '0734579024:  
4990 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9570', 'XX633042010258286181516573', '7233675965:  
4991 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9571', 'XX484915382437141996579468', '1265066247:  
4992 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9572', 'XX405448186269738388404508', '8296622848:  
4993 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9573', 'XX955338931053088901924906', '9099040179:  
4994 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9574', 'XX662761758361432680520775', '3171178331:  
4995 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9575', 'XX4998081216073626571196473', '8086033007:  
4996 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9576', 'XX52971093058771264172007', '45542269780:  
4997 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9577', 'XX15891407850480863147086', '8110437271:  
4998 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9578', 'XX991539646456110567870254', '8999808823:  
4999 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX296393091587170202131236', '96908060468:  
5000 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX7812588898519580806677358', '5541182364:  
5001 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX915670516485388124398147', '2624305478:  
5002
```

Output

#	Time	Action	Message	Duration / Fetch
✓ 5007	20:39:08	INSERT INTO credit_car...	1 row(s) affected	0.000 sec
✓ 5008	20:39:08	INSERT INTO credit_car...	1 row(s) affected	0.000 sec
✓ 5009	20:39:08	INSERT INTO credit_car...	1 row(s) affected	0.000 sec
✓ 5010	20:39:08	INSERT INTO credit_car...	1 row(s) affected	0.000 sec
✓ 5011	20:39:08	INSERT INTO credit_car...	1 row(s) affected	0.000 sec

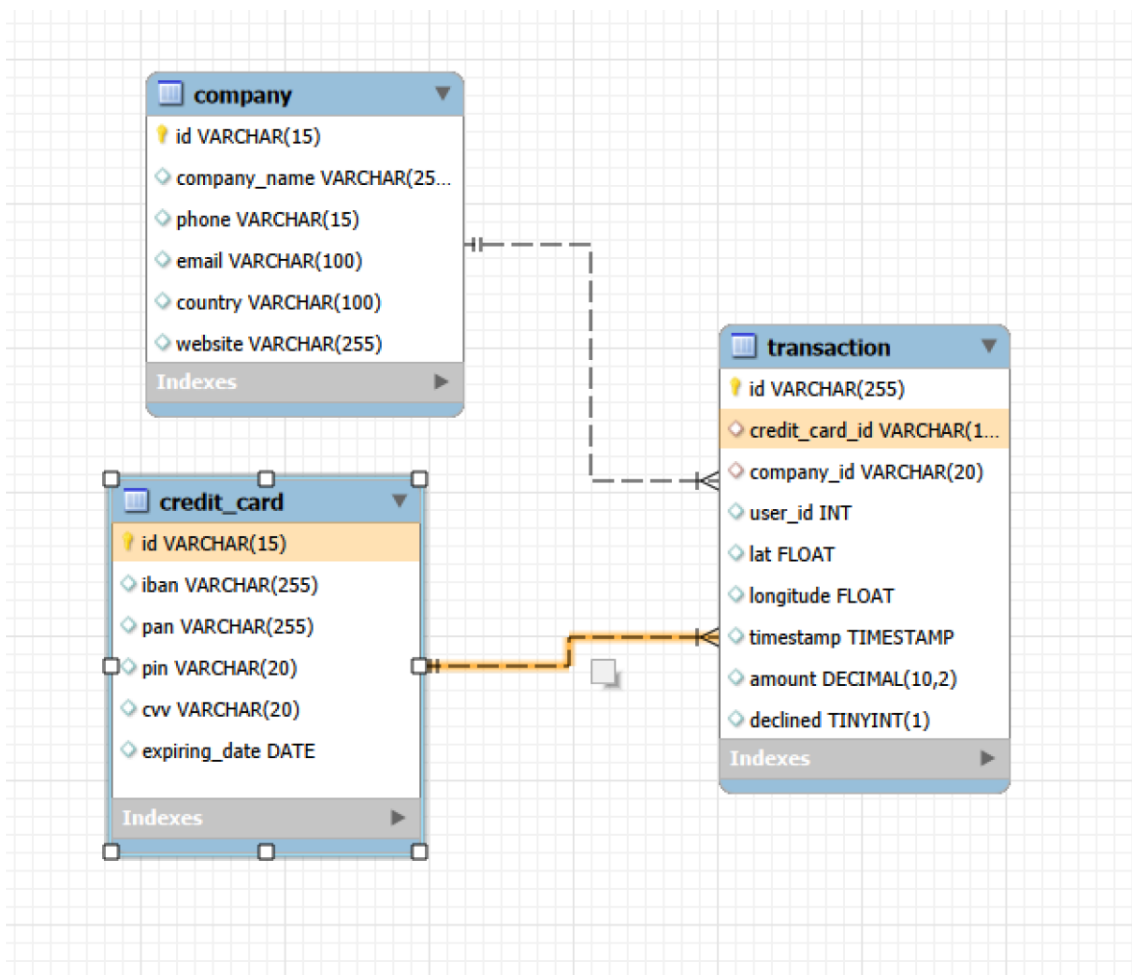
Por último, creamos la relación FOREIGN KEY de credit_card con la tabla transactions:

```
134 # por ultimo, creamos la relacion con la tabla transactions  
135 • ALTER TABLE transaction  
136     ADD CONSTRAINT FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);  
137
```

Output

#	Time	Action	Message
✓ 1	21:01:05	ALTER TABLE transaction ADD CONSTRAINT FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

- En el siguiente diagrama se puede observar un modelo relacional de 3 tablas. La tabla donde ocurren los hechos es “transaction”, ya que registra las operaciones realizadas que tiene que ver con el importe, fecha, localización, usuario, empresa, tarjeta y estado de la transacción. Las tablas “company” y credit card, contienen información descriptiva de las empresas y las tarjetas asociadas.
- Se observa una relación 1 a muchos (1: N) de company hacia transaction y se interpreta que una empresa puede estar asociada a muchas transacciones. La “primary key” en “company” es “id” y está relacionada con “company_id” de “transaction” que actúa como “foreign key”.
- Se observa otra relación 1 a muchos (1: N) de credit_card hacia transaction y se interpreta que una tarjeta puede estar asociada a muchas transacciones. La “primary key” es “id” en “credit_card” y está relacionada con “credit_card_id” en “transaction” que actúa como “foreign key”.



Exercici 5:

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

Identificamos el código de tarjeta:

```
84 • SELECT *
85 FROM transaction AS tr
86 JOIN credit_card AS cr ON cr.id=tr.credit_card_id
87 WHERE credit_card_id='CcU-2938';
```

id	lat	longitude	timestamp	amount	declined	id	iban	pan	pin	cvv	expiring_date
45.4273	-75.6947	2017-12-19 04:58:27	40.43	0	CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30	
45.4214	-75.7026	2024-11-07 12:55:16	291.82	0	CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30	
45.4161	-75.6884	2021-04-05 02:56:06	174.49	0	CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30	
45.4181	-75.7049	2016-12-19 00:48:50	149.70	0	CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30	
45.4281	-75.6915	2019-01-22 18:31:38	273.96	0	CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30	
45.4219	-75.6976	2015-10-03 22:24:28	196.65	0	CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30	
45.4182	-75.7037	2021-09-12 23:03:53	329.20	0	CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30	

Result 8 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5024	11:02:03	SELECT * FROM transac...	1000 row(s) returned	0.000 sec / 0.011

Hacemos el UPDATE modificando la columna Iban:

```
88
89 • UPDATE credit_card
90 SET iban='TR323456312213576817699999'
91 WHERE id='CcU-2938' AND iban = 'TR301950312213576817638661';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	12:44:14	UPDATE credit_card SET iban=TR301950312213576817638661 WHERE id=...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec
2	12:44:29	UPDATE credit_card SET iban=TR323456312213576817699999 WHERE id=...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec

Verificamos que esté correcto el cambio

```
84 • SELECT *
85 FROM transaction AS tr
86 JOIN credit_card AS cr ON cr.id=tr.credit_card_id
87 WHERE credit_card_id='CcU-2938';
```

id	lat	longitude	timestamp	amount	declined	id	iban	pan	pin	cvv	expiring_date
45.4273	-75.6947	2017-12-19 04:58:27	40.43	0	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	
45.4214	-75.7026	2024-11-07 12:55:16	291.82	0	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	
45.4161	-75.6884	2021-04-05 02:56:06	174.49	0	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	
45.4181	-75.7049	2016-12-19 00:48:50	149.70	0	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	
45.4281	-75.6915	2019-01-22 18:31:38	273.96	0	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	
45.4219	-75.6976	2015-10-03 22:24:28	196.65	0	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	
45.4182	-75.7037	2021-09-12 23:03:53	329.20	0	CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	

Result 16 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
2	12:44:29	UPDATE credit_card SET iban=TR323456312213576817699999 WHERE id=...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec
3	12:46:03	SELECT * FROM transaction AS tr JOIN credit_card AS cr ON cr.id=tr.credit_c...	36 row(s) returned	0.000 sec / 0.000 sec

Exercici 6:

En la taula "transaction" ingressa una nova transacció amb la següent informació: Id 108B1D1D-5B23-A76C-55EF-C568E49A99DD, credit_card_id CcU-9999, company_id b-9999, user_id 9999, lat 829.999, longitude -117.999, amount 111.11, declined 0

El primer paso es insertar el id (CcU-9999) en la tabla company el id (CcU-9999) y el iban en la tabla credit card porque no existen, sin esto no puede crearse en la tabla transacción por las dependencias

```
100 # 6) En la taula "transaction" ingressa una nova transacció amb la següent informació:
101 #id 108B1D1D-5B23-A76C-55EF-C568E49A99DD, credit_card_id CcU-9999, company_id b-9999,
102 #user_id 9999, lat 829.999, longitude -117.999, amount 111.11, declined 0
103
104 • INSERT INTO company(id)VALUES('b-9999');
105
106 • INSERT INTO credit_card(id,iban)VALUES('CcU-9999','108B1D1D-5B23-A76C-55EF-C568E49A99DD');
107
108 • INSERT INTO transaction(
109 id,credit_card_id,company_id,user_id,lat,longitude,amount,timestamp,declined)
110 VALUES('108B1D1D-5B23-A76C-55EF-C568E49A99DD','CcU-9999','b-9999','9999','829.999','-117.999','111.11',CURRENT_TIMESTAMP,'0' );
111
112
```

#	Time	Action	Message	Duration / Fetch
1	14:26:37	INSERT IN...	1 row(s) affected	0.015 sec
2	14:26:41	INSERT IN...	1 row(s) affected	0.000 sec

Ahora que lo tenemos en ambas tablas insertamos en transactions todos los valores.

```
100 # 6) En la taula "transaction" ingressa una nova transacció amb la següent informació:
101 #id 108B1D1D-5B23-A76C-55EF-C568E49A99DD, credit_card_id CcU-9999, company_id b-9999,
102 #user_id 9999, lat 829.999, longitude -117.999, amount 111.11, declined 0
103
104 • INSERT INTO company(id)VALUES('b-9999');
105
106 • INSERT INTO credit_card(id,iban)VALUES('CcU-9999','108B1D1D-5B23-A76C-55EF-C568E49A99DD');
107
108 • INSERT INTO transaction(
109 id,credit_card_id,company_id,user_id,lat,longitude,amount,timestamp,declined)
110 VALUES('108B1D1D-5B23-A76C-55EF-C568E49A99DD','CcU-9999','b-9999','9999','829.999','-117.999','111.11',CURRENT_TIMESTAMP,'0' );
111
112
```

#	Time	Action	Message	Duration / Fetch
1	14:29:45	INSERT IN...	1 row(s) affected	0.031 sec

Consultamos y verificamos que este correcto el registro

```
111
112 • SELECT *
113 FROM transaction
114 WHERE credit_card_id='CcU-9999';
```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	2026-09-11 14:29:45	111.11	0

#	Time	Action	Message	Duration / Fetch
5	14:32:19	SELECT id ...	1 row(s) returned	0.016 sec / 0.000 s
6	14:32:27	SELECT * F...	1 row(s) returned	0.000 sec / 0.000 s

Exercici 7:

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card.
Recorda mostrar el canvi realitzat.

Eliminamos la columna pan en la tabla credit_card

```
117 #Exercici 7
118 #Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card.
119 #Recorda mostrar el canvi realitzat.
120
121 • ALTER TABLE credit_card
122 DROP COLUMN pan;
123
124
125
126
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 1	15:56:21	ALTER TAB...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.172 sec

Verificamos que ya no exista.

```
167 #verificamos que exista la columna pan
168 • SELECT *
169 FROM credit_card;
170
171 #borramos la columna pan
172 • ALTER TABLE credit_card
173 DROP COLUMN pan;
174
175 #comprobamos de nuevo que se haya hecho el cambio
176 • SELECT *
177 FROM credit_card;
178
```

Result Grid					
Filter Rows:					
Edit: Export/Import: Wrap Cell Content: Fetch rows:					
	id	iban	pin	cvv	expiring_date
▶	CcS-4857	XX4857591835292505850771	1819	467	2025-09-27
	CcS-4858	XX8581768137002436094025	3964	817	2028-12-28
	CcS-4859	XX7826930491423553609370	4983	277	2026-11-26
	CcS-4860	XX5559590368835304645299	6876	661	2027-07-27
	CcS-4861	XX2035182877195191627307	5710	398	2026-04-25

credit_card 22 x

Output

Action Output

#	Time	Action	Message
✓ 1	19:26:26	SELECT * FROM credit_card	5001 row(s) returned

Exercici 8:

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

La taula de products.csv l'utilitzarem més endavant.

Vamos a crear 2 bases de datos, una para la fase staging que funcionara como backup 'data' y otra donde irán las dimensiones y tabla de hechos 'bank_transactions' que previamente había creado (por eso el warning).

```
187 #CREAMOS EL SCHEMA data donde estaran almacenadas nuestras tablas de carga y backup
188 • CREATE SCHEMA IF NOT EXISTS data;
189
190 • USE data;
191
```

#	Time	Action	Message	Duration / Fi
1	19:28:46	CREATE SCHEMA IF NOT EXISTS data	1 row(s) affected, 1 warning(s): 1007 Can't create database 'data'; databa...	0.031 sec
2	19:28:46	USE data	0 row(s) affected	0.000 sec

Entonces, primer paso es crear las tablas de staging y cargar los datos que estan en formato CSV. Asi que creamos una a una las tablas. Lo primero que deberíamos habernos percatado, es que “american_users” y “european_users”, tienen la misma estructura y como necesitamos crear un modelo estrella, lo ideal es unir las como una sola tabla de usuarios

```
191 #creamos la tabla users
192 • CREATE TABLE IF NOT EXISTS data.staging_users (
193     id INT,
194     name VARCHAR(255),
195     surname VARCHAR(255),
196     phone VARCHAR(255),
197     email VARCHAR(255),
198     birth_date VARCHAR(50),
199     country VARCHAR(255),
200     city VARCHAR(255),
201     postal_code VARCHAR(255),
202     address VARCHAR(255),
203     signup_date VARCHAR(50),
204     user_segment VARCHAR(255),
205     income_band VARCHAR(255)
206 );
207
208 #creamos la tabla companies
209
210 #creamos la tabla companies
211 • CREATE TABLE IF NOT EXISTS data.stagin
212     company_id VARCHAR(20),
213     company_name VARCHAR(255),
214     phone VARCHAR(255),
215     email VARCHAR(255),
216     country VARCHAR(255),
217     website VARCHAR(255),
218     merchant_category VARCHAR(255),
219     merchant_price_position VARCHAR(25
220 );
221
222 #creamos la tabla credit cards
223 • CREATE TABLE IF NOT EXISTS data.stagin
224     id VARCHAR(20),
225     user_id INT,
226     iban VARCHAR(255),
227     pan VARCHAR(255),
228     pin VARCHAR(255),
229     cvv VARCHAR(255),
230     track1 VARCHAR(255),
231     track2 VARCHAR(255),
232     expiring_date VARCHAR(255),
233     card_type VARCHAR(255),
234     card_renewal_flag BOOLEAN
235 );
```

#	Time	Action	Message
1	17:32:51	DROP TABLE 'data'.staging_american_users	0 row(s) affected
2	17:32:55	DROP TABLE 'data'.staging_european_users	0 row(s) affected
3	17:34:21	CREATE TABLE IF NOT EXISTS data.staging_users (id INT, name VARCHAR(255), surname ...	0 row(s) affected

El siguiente paso es insertar los datos del CSV en las tablas creadas, para ello usaremos la función de LOAD DATA INFILE, la única carpeta habilitada para estas cargas es la carpeta uploads donde está instalado MYSQL server, “C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/tu_archivo.csv”

```
275 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'
276 INTO TABLE DATA.staging_users
277 FIELDS TERMINATED BY ','
278 ENCLOSED BY '"'
279 LINES TERMINATED BY '\n'
280 IGNORE 1 ROWS;
```

Hacemos lo mismo para el resto de tablas, hasta llegar a la de transactions. La diferencia con esta carga, es que este CSV está separado por “;” en vez de por “,” así que lo modificamos, sino dará error.

```
130
131 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
132 INTO TABLE DATA.staging_transactions
133 FIELDS TERMINATED BY ';'
134 ENCLOSED BY ''
135 LINES TERMINATED BY '\n'
136 IGNORE 1 ROWS;
137
```

NOTA: una vez cargados todos los CSV que nos piden, nos aseguramos que la información cargada corresponda con el número de filas y columnas de los ficheros CSV.

```
115 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv'
116 INTO TABLE DATA.staging_companies
117 FIELDS TERMINATED BY ','
118 ENCLOSED BY '"'
119 LINES TERMINATED BY '\n'
120 IGNORE 1 ROWS;
121
122 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
123 INTO TABLE DATA.staging_credit_cards
124 FIELDS TERMINATED BY ','
125 ENCLOSED BY '"'
126 LINES TERMINATED BY '\n'
127 IGNORE 1 ROWS;
128
129
130 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
131 INTO TABLE DATA.staging_transactions
132 FIELDS TERMINATED BY ';'
133 ENCLOSED BY ''
134 LINES TERMINATED BY '\n'
135 IGNORE 1 ROWS;
136
137
```

Output

#	Time	Action	Message
1	23:50:21	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv' INTO TABLE DATA.staging_american_users FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	1010 row(s) affected Records
2	23:50:21	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv' INTO TABLE DATA.staging_european_users FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	3990 row(s) affected Records
3	23:50:21	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv' INTO TABLE DATA.staging_companies FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	100 row(s) affected Records
4	23:50:21	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv' INTO TABLE DATA.staging_credit_cards FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	5000 row(s) affected Records
5	23:50:21	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv' INTO TABLE DATA.staging_transactions FIELDS TERMINATED BY ';' ENCLOSED BY '' LINES TERMINATED BY '\n' IGNORE 1 ROWS	
6	23:50:34	SELECT * FROM data.staging_transactions	100000 row(s) returned

Ahora que ya tenemos los CSV cargados, crearemos nuestra tabla “**bank_transactions**” donde estarán las dimensiones y los hechos.

```
148 • CREATE DATABASE IF NOT EXISTS bank_transactions;
149 • USE bank_transactions;
150
151
152
```

Output

#	Time	Action
1	23:54:21	CREATE DATABASE IF NOT EXISTS bank_transactions
2	23:54:21	USE bank_transactions

El objetivo es que los inserts provengan de las tablas staging que hemos creado antes en la base de datos “data”

Procedemos a crear la tabla “**dim_users**”, la PRIMARY KEY es “id”, el resto de campos son descriptivos con lo cual van en VARCHAR y los campos de fecha como DATE.

```
177 #Creamos la nueva tabla de dim_users
178 CREATE TABLE dim_users (
179   id INT PRIMARY KEY,
180   name VARCHAR(255),
181   surname VARCHAR(255),
182   phone VARCHAR(255),
183   email VARCHAR(255),
184   birth_date DATE,
185   country VARCHAR(255),
186   city VARCHAR(255),
187   postal_code VARCHAR(255),
188   address VARCHAR(255),
189   signup_date DATE,
190   user_segment VARCHAR(255),
191   income_band VARCHAR(255) );
192
```

Output

#	Time	Action
1	00:04:00	CREATE TABLE dim_users (id INT PRIMARY KEY, name

El último paso con esta tabla es hacer el INSERT. Para ello INSERT INTO. Aquí haremos la transformación de los campos fecha con STR_TO_DATE

```
345 # hacemos un insert y vamos a extraerlos de la BD data.staging_users que es donde cargamos nuestros csv
346 INSERT INTO dim_users(
347   id,name,surname,phone,email,birth_date,country,city,postal_code,address,signup_date,user_segment,income_band
348 )
349 SELECT id,name,surname,phone,email,STR_TO_DATE(birth_date, '%b %d, %Y'),country,city,postal_code,address,STR_TO_DATE(signup_date, '%Y-%m-%d'),user_segment,income_band
350 FROM data.staging_users;
351
352
353
```

Output

#	Time	Action	Message
1	18:15:16	CREATE TABLE IF NOT EXISTS dim_users (id INT PRIMARY KEY, name VARCHAR(255), surname V...	0 row(s) affected
2	18:15:23	INSERT INTO dim_users(id,name,surname,phone,email,birth_date,country,city,postal_code,address,si...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Ahora seguimos con **dim_companies**. Su clave primaria es company_id y el resto de campos descriptivos como VARCHAR.

```
62 CREATE TABLE dim_companies
63 (
64   company_id VARCHAR(20) PRIMARY KEY,
65   company_name VARCHAR(255),
66   phone VARCHAR(20),
67   email VARCHAR(255),
68   country VARCHAR(255),
69   website VARCHAR(255),
70   merchant_category VARCHAR(255),
71   merchant_price_position VARCHAR(255)
72 );
73
```

Form Editor | Navigate: 3 / 100

Company_id: b-2230

Fusce Corp.

companies 12 x

Output

#	Time	Action	Message	Duration / Fetch
63	22:10:04	CREATE TABLE dim_com...	0 row(s) affected	0.016 sec

insertamos los datos en dim_companies con sus valores.

```
75 • INSERT INTO dim_companies
76 SELECT
77 company_id,company_name,phone,email,country,website,merchant_category,merchant_price_position
78 FROM companies;
79
80
81
82
83
84
85
86
87
88
89
90
```

Output

#	Time	Action	Message	Duration / Fetch
65	22:11:46	SELECT company_id,com...	100 row(s) returned	0.000 sec / 0.000 sec
66	22:15:09	INSERT INTO dim_compa...	Error Code: 1136. Column count doesn't match value count at row 1	0.000 sec
67	22:15:35	INSERT INTO dim_compa...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0	0.000 sec

Mismo proceso con **dim_credit_card**, la primary key es “id”, campos descriptivos como VARCHAR, user_id INT, card_renewal_flag como Boolean, expiring_date como date y luego aplicamos el STR_TO_DATE en el insert

```
---
228 #creamos dim_credit_card con su primary key
229 • CREATE TABLE dim_credit_card (
230     id VARCHAR(15) PRIMARY KEY,
231     user_id INT,
232     iban VARCHAR(255),
233     pan VARCHAR(255),
234     pin VARCHAR(10),
235     cvv VARCHAR(10),
236     track1 VARCHAR(255),
237     track2 VARCHAR(255),
238     expiring_date DATE,
239     card_type VARCHAR(50),
240     card_renewal_flag BOOLEAN
241 );
242
243 #hacemos el insert de los datos consultados en dim credit card
244 • INSERT INTO dim_credit_card
245 SELECT id,user_id,iban,pan,pin,cvv,track1,track2,STR_TO_DATE(expiring_date, '%m/%d/%y'),card_type,card_renewal_flag
246 FROM data.staging_credit_cards;
247
248
249
```

Output

#	Time	Action	Message
1	00:13:08	CREATE TABLE dim_credit_card (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(255), pan VARCHAR(255), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(255), track2 VARCHAR(255), expiring_date DATE, card_t...	0 row(s) affected
2	00:13:08	INSERT INTO dim_credit_card SELECT id,user_id,iban,pan,pin,cvv,track1,track2,STR_TO_DATE(expiring_date, '%m/%d/%y'),card_type,card_renewal_flag FROM data.staging_credit_cards	5000 row(s) affected

La última, en crear es **fact_transactions** la cual será la que almacene los hechos y operaciones. Añadiremos las cláusulas FOREIGN KEY que relaciona con las dimensiones. Adicionalmente, se ha creado 2 campos, fecha que extraen el día “tr_date” y la hora “tr_time” de la columna original que combina ambas “timestamp” , la PRIMARY KEY es “id” y FOREIGN KEY, company_id y card_id como INT y VARCHAR, campos decimales que tienen que ver con “amount”, lat, longitude y el resto VARCHAR.

```

251
252      #creamos fact_transactions con su primary key
253  CREATE TABLE fact_transactions (
254      id VARCHAR(50) PRIMARY KEY,
255      company_id VARCHAR(20),
256      card_id VARCHAR(15),
257      user_id INT,
258      product_ids VARCHAR(100),
259      campaign_id VARCHAR(100),
260      timestamp DATETIME, #Fecha original datetime
261      tr_date DATE, #Campo adicional fecha
262      tr_time TIME, #Campo adicional hora
263      amount DECIMAL(10,2),
264      declined BOOLEAN,
265      lat DECIMAL(10,6),
266      longitude DECIMAL(10,6),
267      discount_amount DECIMAL(10,2),
268      tax_amount DECIMAL(10,2),
269      shipping_amount DECIMAL(10,2),
270      channel VARCHAR (100),
271      device_type VARCHAR (100),
272      is_international BOOLEAN,
273      decline_reason VARCHAR(100),
274      distance_km DECIMAL(10,2),
275
276      #creamos las relaciones con las tablas y sus referencias
277      FOREIGN KEY (company_id) REFERENCES dim_companies(id),
278      FOREIGN KEY (card_id) REFERENCES dim_credit_card(id),
279      FOREIGN KEY (user_id) REFERENCES dim_users(id)
280  );

```

Output		
#	Time	Action
✓ 1	01:06:40	CREATE TABLE fact_transactions (id VARCHAR(50) PRIMARY KEY, company_id VARCHAR(20), card_id VARCHAR(15),

Insertamos y confirmamos que estén los 100.000 registros

```

283  #creamos el insert de los datos consultados en dim transactions
284  INSERT INTO fact_transactions (
285      id, company_id, card_id, user_id, product_ids, campaign_id,
286      timestamp, tr_date, tr_time, amount, declined, lat, longitude,
287      discount_amount, tax_amount, shipping_amount, channel, device_type, is_international, decline_reason, distance_km
288  )
289  SELECT id, business_id, card_id, user_id, product_ids, campaign_id,
290      timestamp, DATE(timestamp) AS tr_date, TIME(timestamp) AS tr_time, amount, declined, lat, longitude,
291      discount_amount, tax_amount, shipping_amount, channel, device_type, is_international, decline_reason, distance_km
292  FROM data_staging.transactions;
293
294  #verificamos los datos
295  SELECT * FROM bank_transactions, fact_transactions;
296

```

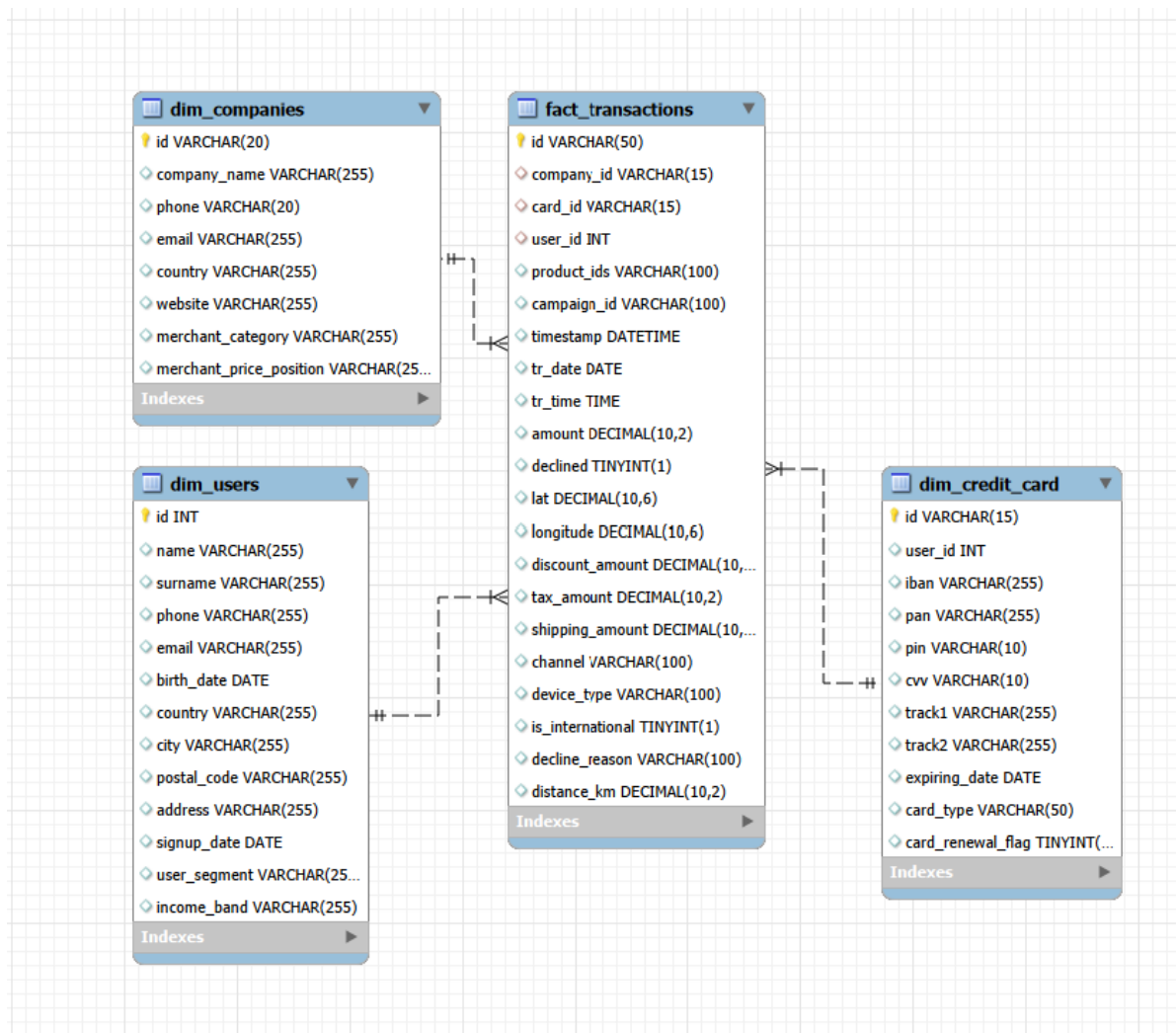
id	company_id	card_id	user_id	product_ids	campaign_id	timestamp	tr_date	tr_time	amount	declined	lat	longitude	discount_amount	tax_amount	shipping_amount	channel	device_type	is_international	decline_reason	distance_km
00043440-3048-4048-4300-438AE8B1000	b-2458	Cid-4284	4713	16, 26, 67, 87	SUMMER_TRAVEL	2024-08-28 07:16:46	2024-08-28	07:16:46	433.51	0	46.199920	1.435540	34.17	72.25	0.00	online	mobile	0		50.65
000440FE-8630-403F-830E-C70E07E0CA0D	b-2370	Cid-9119	4308	66, 86, 87	CHRISTMAS_PEAK	2016-12-21 20:07:18	2016-12-21	20:07:18	172.56	0	41.987206	12.221760	18.92	26.86	6.99	online	tablet	0		494.51
00045006-003E-4F3F-838E-C5E074E87500	b-2390	Cid-6699	2118	30, 11, 16, 81	SUMMER_TRAVEL	2020-07-14 15:37:45	2020-07-14	15:37:45	332.38	0	29.757296	-95.179637	16.10	22.47	0.00	store	pos	1		1157.10
000481C3-3C3E-4F2F-83AD-42CE0B0048B0	b-2230	Cid-6696	2115	72	NO_CAMPAIGN	2017-09-04 19:44:53	2017-09-04	19:44:53	175.00	0	53.548884	-113.503053	1.05	14.45	0.00	store	pos	0		540.32
00051AA4-9C3E-42A8-8070-C3802A183E2	b-2266	Cid-7606	3025	18	WINTER_SALE	2017-01-05 18:19:25	2017-01-05	18:19:25	158.55	0	52.288370	1.690806	25.31	25.56	8.99	marketplace	desktop	0		28.32
00068112-02FE-44FF-8C09-EDC3C2B3CA4D	b-2598	Cid-3308	215	35, 35, 19	NO_CAMPAIGN	2022-09-23 04:51:43	2022-09-23	04:51:43	317.97	0	53.553466	113.469124	2.87	26.25	0.00	store	pos	0		539.82
0008A151-80CF-4E31-9053-A468FF77AA0	b-2546	Cid-7509	2028	93, 55, 28, 91	NO_CAMPAIGN	2022-12-31 00:06:36	2022-12-31	00:06:36	457.51	0	52.522484	13.552942	5.52	79.40	0.00	online	mobile	1		562.79
0008D494-6245-4DFF-955D-2C084191CFB8	b-2526	Cid-8483	3602	55, 6, 72	NO_CAMPAIGN	2017-07-18 07:52:02	2017-07-18	07:52:02	225.49	0	45.402009	-73.570637	2.85	17.55	12.99	online	tablet	0		1.11
000A12EC-C38E-4482-4610-73DAB814A362	b-2558	Cid-4467	1886	46, 26, 73	BACK_TO_SCHOOL	2018-09-08 05:29:58	2018-09-08	05:29:58	374.41	0	35.742477	-3.300194	27.93	62.40	0.00	store	pos	0		233.85
000A12F4-1414-4080-9053-9E30A4D958E2	b-2550	Cid-5966	1385	6, 89, 19	BACK_TO_SCHOOL	2020-09-17 04:02:19	2020-09-17	04:02:19	378.00	0	52.082159	5.284039	57.31	65.60	0.00	online	desktop	0		5.63
000A3879-3472-41D9-AF60-42D35038543C	b-2590	Cid-4569	42	58, 61, 55	NO_CAMPAIGN	2020-02-07 23:03:45	2020-02-07	23:03:45	185.46	0	39.949023	-75.171936	1.62	11.66	12.99	online	mobile	0		0.70
000A5ED4-F08E-4146-804A-5A2FC7311007	b-2426	Cid-8134	3553	62	SUMMER_TRAVEL	2017-07-10 07:11:41	2017-07-10	07:11:41	187.69	0	49.374779	-122.368481	20.41	15.17	3.99	online	mobile	1		1358.59

fact_transactions 22 x

#	Time	Action
1	01:13:45	INSERT INTO fact_transactions (id, company_id, card_id, user_id, product_ids, campaign_id, timestamp, tr_date, tr_time, amount, declined, lat, longitude, discount_amount, tax_amount, shipping_amount, channel, device_type, is_international, decline_reason, distance_km) SE.
2	01:13:49	SELECT * FROM bank_transactions, fact_transactions

100000 rows returned

- En el siguiente diagrama, se puede observar un modelo estrella con 3 dimensiones dim_companies, dim_credit_card y dim_users. Aquí se almacenan los datos descriptivos. En el centro, se encuentra la tabla de hechos fact_transactions, donde se almacenan las transacciones o hechos.
- La tabla fact_transactions, se relaciona con las dimensiones mediante claves foráneas: company_id, card_id y user_id, que hacen referencia a las claves primarias de las tablas de dimensiones.
- Se observan 3 relaciones, de uno a muchos (1:N) desde las dimensiones, hacia la tabla de hechos. Se comprende como una empresa, una tarjeta de crédito y un usuario. Puedes tener múltiples transacciones mientras que cada transacción pertenece a una empresa, una tarjeta y un usuario.



Exercici 9:

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
444 #Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
445 SELECT
446     (SELECT u.name
447      FROM dim_users AS u
448      WHERE u.id = tr.user_id) AS usuario,
449      COUNT(*) AS n_transacciones
450 FROM fact_transactions AS tr
451 WHERE EXISTS (
452     SELECT user_id
453     FROM fact_transactions AS t
454     WHERE t.user_id = tr.user_id
455     GROUP BY t.user_id
456     HAVING COUNT(*) > 80
457 )
458 GROUP BY tr.user_id
459 ORDER BY n_transacciones DESC;
```

usuario	n_transacciones
Molly	110
Dixwi	94
Bnyr	91
Sfzzoh	81

#	Time	Action	Message
3	19:46:09	SELECT CONCAT((SELECT u.name FROM dim_users A...	4 row(s) returned

Exercici 10:

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```
464 #Exercici 10
465 #Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.
466 SELECT cr.iban, ROUND(AVG(amount),2) AS media_gasto
467 FROM fact_transactions AS tr
468 JOIN dim_credit_card AS cr ON cr.id=tr.card_id
469 JOIN dim_companies AS c ON c.id=tr.company_id
470 WHERE c.company_name = "Donec Ltd"
471 AND tr.declined = 0
472 GROUP BY cr.iban
473 ORDER BY media_gasto DESC;
```

iban	media_gasto
US212768278745981809500233	708.14
CA817443448587215122047864	690.81
US157275989571099139868887	647.08
US594988240619959562749388	638.82
US186466262266569948042760	629.76
US572538468703418440090624	626.89

#	Time	Action	Message
1	19:56:04	SELECT cr.iban, ROUND(AVG(amount),2) AS media_gasto FROM fact_...	367 row(s) returned

NIVEL 2

Exercici 1:

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes.

#Mostra la data de cada transacció juntament amb el total de les vendes.

```
481 #Exercici 1
482 #Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes.
483 #Mostra la data de cada transacció juntament amb el total de les vendes.
484
485 • SELECT tr_date AS fecha, ROUND(SUM(amount),2) as total_ingresos_dia
486 FROM fact_transactions AS tr
487 JOIN dim_companies AS c ON c.id=tr.company_id
488 WHERE declined = 0
489 GROUP BY tr_date
490 ORDER BY total_ingresos_dia DESC
491 LIMIT 5;
```

fecha	total_ingresos_dia
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

Result 32 x

Output

#	Time	Action	Message
3	19:59:06	SELECT tr_date AS fecha, SUM(amount) as total_ingresos_dia FROM...	5 row(s) returned

Exercici 2:

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor

#comprès entre 350 i 400 euros i en alguna d'aquestes dates:

#29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```
495 #Exercici 2
496 #Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor
497 #comprès entre 350 i 400 euros i en alguna d'aquestes dates:
498 #29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.
499
500 • SELECT c.company_name AS empresa,c.phone,c.country AS pais,tr.tr_date as fecha, tr.amount AS monto_transaccion
501 FROM fact_transactions AS tr
502 JOIN dim_companies AS c ON c.id=tr.company_id
503 WHERE tr.amount BETWEEN 350 AND 400
504 AND tr.tr_date IN('2015-04-29', '2018-07-20', '2024-03-13')
505 AND declined = 0
506 ORDER BY monto_transaccion DESC;
```

empresa	phone	pais	fecha	monto_transaccion
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Piede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

Result 36 x

Output

#	Time	Action	Message
1	20:02:04	SELECT c.company_name AS empresa,c.phone,c.country AS pais,tr.tr_...	6 row(s) returned

Exercici 3:

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

```
504 #Exercici 3
505 #Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi,
506 #per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses,
507 #però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis
508 #si tenen igual o més de 400 transaccions o menys.
509
510 • SELECT c.company_name, COUNT(*) AS cantidad_transacciones,
511 CASE WHEN COUNT(*)>400 THEN '>= 400' ELSE '< 400' END AS transacciones
512 FROM fact_transactions AS tr
513 JOIN dim_companies AS c ON c.id=tr.company_id
514 GROUP BY tr.company_id
515 ORDER BY transacciones;
```

company_name	cantidad_transacciones	transacciones
Nec Luctus LLC	399	< 400
Fringilla LLC	397	< 400
Lorem Eu Incorporated	380	< 400
Ac Fermentum Incorporated	2401	>= 400
Magna A Neque Industries	410	>= 400
Fusce Corp.	447	>= 400
Conwallis In Incorporated	1514	>= 400
Ante Iaculis Nec Foundation	472	>= 400

Result 152 x

Output

Action Output

#	Time	Action	Message
1	19:23:49	SELECT c.company_name, COUNT(*) AS cantidad_transacciones, CASE WHEN COUNT(*)>400 THEN '>= 400' ELSE '< 400' END AS transacciones	100 row(s) returned

Exercici 4:

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

```
523 #Exercici 4
524 #Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.
525
526 #primero comprobamos que este el registro
527 • SELECT * FROM fact_transactions WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
528
529 #Ahora procedemos a eliminar
530 • DELETE FROM fact_transactions
531 WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
532
533 #comprobamos que se haya eliminado
534 • SELECT * FROM fact_transactions WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
535
536 #Al eliminar un registro de fact_transactions, unicamente se elimina la transacción correspondiente.
```

id	company_id	card_id	user_id	product_ids	campaign_id	timestamp	tr_date	tr_time	amount	declined	lat	lo
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

fact_transactions 38 x

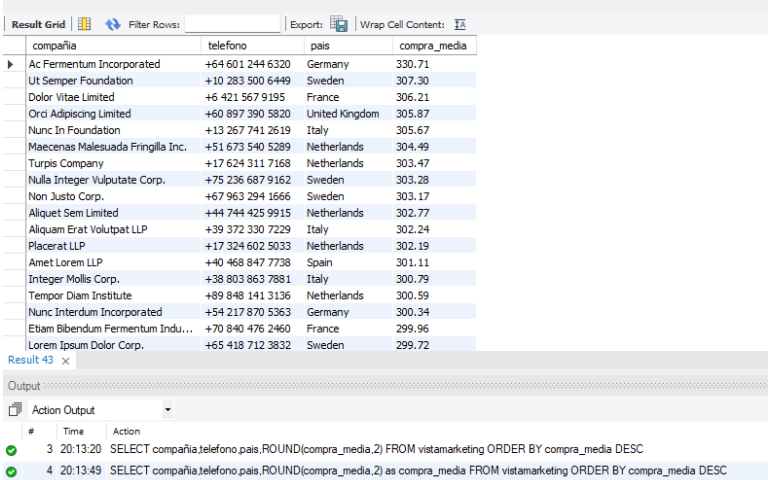
Output

Action Output

#	Time	Action	Message
1	20:04:50	SELECT * FROM fact_transactions WHERE id='000447FE-B650-4DCF-...	0 row(s) returned

Exercici 5:

```
543 #Exercici 5
544 #La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives.
545 #S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions.
546 #Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació:
547 #Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia.
548 #Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.
549
550 #cremamos la vista
551 • CREATE VIEW VistaMarketing AS
552 SELECT c.company_name AS compaia,c.phone AS telefono,c.country AS pais,
553 AVG(tr.amount) AS compra_media
554 FROM fact_transactions AS tr
555 JOIN dim_companies AS c ON c.id=tr.company_id
556 WHERE tr.declined = 0
557 GROUP BY tr.company_id;
558
559 #Consultamos ordenando de mayor a menor compra_media
560 • SELECT compaia,telefono,pais,ROUND(compra_media,2) as compra_media
561 FROM vistamarketing
562 ORDER BY compra_media DESC;
```



compaia	telefono	pais	compra_media
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.71
Ut Semper Foundation	+10 283 500 6449	Sweden	307.30
Dolor Vitae Limited	+6 421 567 9195	France	306.21
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	305.87
Nunc In Foundation	+13 267 741 2619	Italy	305.67
Maecenas Malesuada Fringilla Inc.	+51 673 540 5289	Netherlands	304.49
Turpis Company	+17 624 311 7168	Netherlands	303.47
Nulla Integer Vulputate Corp.	+75 236 687 9162	Sweden	303.28
Non Justo Corp.	+67 963 294 1666	Sweden	303.17
Aliquet Sem Limited	+44 744 425 9915	Netherlands	302.77
Aliquam Erat Volutpat LLP	+39 372 330 7229	Italy	302.24
Placerat LLP	+17 324 602 5033	Netherlands	302.19
Amet Lorem LLP	+40 468 847 7738	Spain	301.11
Integer Mollis Corp.	+38 803 863 7881	Italy	300.79
Tempor Diam Institute	+89 848 141 3136	Netherlands	300.59
Nunc Interdum Incorporated	+54 217 870 5363	Germany	300.34
Etiam Bibendum Fermentum Indu...	+70 840 476 2460	France	299.96
Lorem Ipsum Dolor Corp.	+65 418 712 3832	Sweden	299.72

NIVEL 3

Exercici 1:

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

Para este ejercicio usamos una CTE (tabla temporal) que nos permite usar funciones de ventana, la función ROW_NUMBER permite la partición de un campo en este caso card_id, ordenado por fecha en orden descendente, ya que necesitamos las ultimas 3 transacciones.

cerramos la tabla temporal y la anidmamos con el siguiente SELECT, para comprobar el estado de tarjeta usaremos UN CASE, esta función nos permite tomar como referencia los valores de un campo y darle una condición para convertirlo en columna, tomamos el campo declined y lo sumamos, en el caso que la suma sea = a 3 es “inactivo”, caso contrario “activo” y renombramos con estado_tarjeta.

```

599 WITH transacciones_tarjeta AS( #usamos una common table expression (tabla temporal) que nos permite usar funciones de ventana
600 SELECT tr.card_id,tr.declined,tr.timestamp,
601        ROW_NUMBER() OVER(      # la funcion ROW_NUMBER permite la particion de un campo en este caso card_id
602          PARTITION BY card_id ORDER BY timestamp DESC #ordenado por fecha en orden descendente, ya que queremos las ultimas 3 transacciones
603        ) AS partition
604 FROM fact_transactions tr
605 ) # cerramos la tabla temporal y la usaremos en el siguiente select
606 SELECT card_id,
607        #usamos un case para crear una columna a partir de la condicion de la suma de declined, si la suma de declined es igual a 3 entonces inactivo caso contrario activo
608        CASE WHEN SUM(declined) = 3 THEN 'inactivo'ELSE 'activo'END AS estado_tarjeta
609 FROM transacciones_tarjeta
610 WHERE partition <=3 # llamamos al alias de la funcion row_number y aplicamos la condición menor o igual a 3 transacciones
611 GROUP BY card_id;

```

card_id	estado_tarjeta
CcS-4857	activo
CcS-4858	activo
CcS-4859	activo
CcS-4860	activo
CcS-4861	activo
CcS-4862	activo
CcS-4863	activo
CcS-4864	activo
CcS-4865	activo
CcS-4866	activo
CcS-4867	activo
CcS-4868	activo
CcS-4869	activo
CcS-4870	activo
CcS-4871	activo
CcS-4872	activo
CcS-4873	activo

Result: 46 x

Output

#	Time	Action	Message
1	20:23:07	WITH transacciones_tarjeta AS(#usamos una common table expression (tabla temporal) que nos permite usar funciones de ventana SELECT tr.card_id,tr.timestamp, ROW_NUMBER() OVER(...	5000 row(s) returned

Para finalizar llamamos a nuestra tabla temporal (partición) en el WHERE y le decimos que muestre todos los valores <= 3.

Ahora que sabemos los datos, creamos nuestra Base de datos estado_tarjetas_credito

```

579 #Una vez ya sabemos los datos ahora si creamos la tabla con las buenas practicas
580 CREATE TABLE IF NOT EXISTS estado_tarjetas_credito (
581     card_id VARCHAR(50) PRIMARY KEY,
582     estado_tarjeta VARCHAR(20),
583     total_operaciones_declinadas INT,
584     FOREIGN KEY (card_id) REFERENCES dim_credit_card(id) #añadimos la referencia con la dim_credit_card
585 );

```

Output

#	Time	Action	Message
1	19:53:34	CREATE TABLE IF NOT EXISTS estado_tarjetas_credito (card_id VARCHAR(50) PRIMARY KEY, ...	0 row(s) affected

Realizamos el insert en estado_tarjetas_credito usando la misma query

```

223 #Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declined entonces es inactivo,
224 #si por lo menos una no es rechazada entonces es activo. Partiendo de esta tabla responde:
225 # ¿Cuántas tarjetas están activas?
226
227 INSERT INTO estado_tarjetas_credito (card_id, estado_tarjeta, total_operaciones_declinadas) #Hacemos el insert a traves de la consulta
228 WITH transacciones_tarjeta AS( #usamos una common table expression (tabla temporal) que nos permite usar funciones de ventana
229 SELECT tr.card_id,tr.declined,tr.timestamp,
230        ROW_NUMBER() OVER(      # la funcion ROW_NUMBER permite la particion de un campo en este caso card_id
231          PARTITION BY card_id ORDER BY timestamp DESC #ordenado por fecha en orden descendente, ya que queremos las ultimas 3 transacciones
232        ) AS partition
233 FROM fact_transactions tr
234 ) # cerramos la tabla temporal y la usaremos en el siguiente select
235 SELECT card_id,
236        #usamos un case para crear una columna a partir de la condicion de la suma de declined, si la suma de declined es igual a 3 entonces inactivo caso contrario activo
237        CASE WHEN SUM(declined) = 3 THEN 'inactivo'ELSE 'activo'END AS estado_tarjeta,
238        SUM(declined) AS total_operaciones_declinadas #usamos las operaciones declined
239 FROM transacciones_tarjeta
240 WHERE partition <=3 # llamamos al alias de la funcion row_number y aplicamos la condición menor o igual a 3 transacciones
241 GROUP BY card_id;
242

```

Output

#	Time	Action	Message	Duration / Sec
1	22:47:16	CREATE TABLE IF NOT EXISTS estado_tarjetas_credito (card_id VARCHAR(50) PRIMARY KEY, estado_tarjeta VARCHAR(20), total_operaciones_declinadas ...	0 row(s) affected	0.032 sec
2	22:48:51	INSERT INTO estado_tarjetas_credito (card_id, estado_tarjeta, total_operaciones_declinadas) #Hacemos el insert a traves de la consulta WITH transacciones_tarjeta AS(...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.329 sec

Luego verificamos el número de tarjetas activas, en este caso las 5000 están activas.



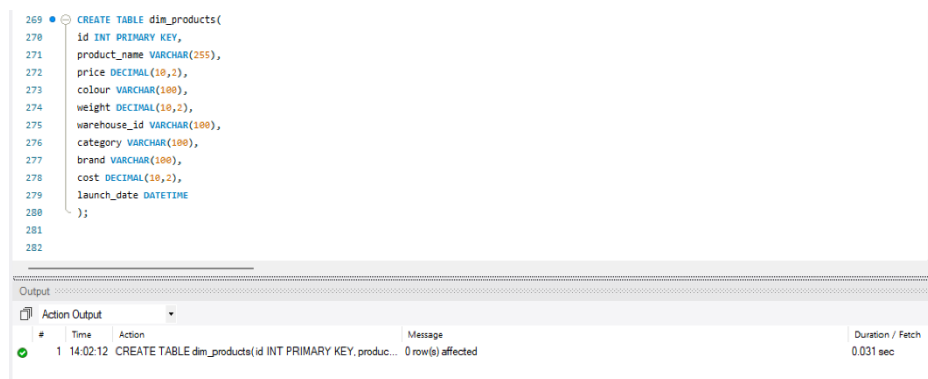
Exercici 2:

#Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podem fer-ho),

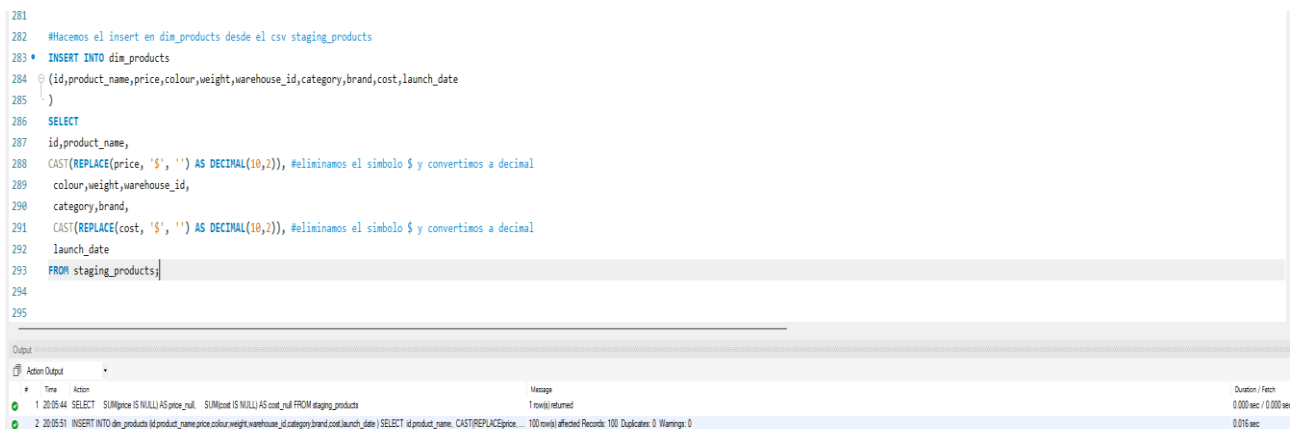
#tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

El primer paso sería crear la tabla dim_products, sin esta tabla no puede existir la tabla puente.



Posteriormente hacemos el INSERT, como tiene 2 campos, Price y cost, con un símbolo que sql no reconoce, lo convertimos con CAST para que queden los números decimales limpios.



Ahora toca normalizar la columna producto_ids de la fact_transactions, como debemos hacerlo en sql, Extraemos los ids de la columna con la función JSON_TABLE ()

```

655
656 # Extraemos los ids de la columna product_ids en fact_transactions con la funcion JSON_TABLE()
657 SELECT
658   tr.id AS transaction_id,
659   jt.product_id
660 FROM fact_transactions tr
661 JOIN JSON_TABLE(
662   CONCAT('[', tr.product_ids, ']'), '$[*]' COLUMNS (product_id INT PATH '$')
663   # CONCAT('[', t.product_ids, ']') Añade [ ] con concat porque la funcion JSON lee arrays
664   # '$[*]' el $ sera nuestra columna 't.product_ids' y con [*] recorre todos los elementos del array
665   # COLUMNS extrae cada elemento y lo convierte a columna
666   # (product_id INT PATH '$') lo renombras a product_id, guárdalo como INT de lo extraido con '$' = t.prod
667 ) AS jt; # referenciamos la funcion como tabla
668

```

transaction_id	product_id
00043A49-2949-494B-A5DD-A5BAE38B19DD	16
00043A49-2949-494B-A5DD-A5BAE38B19DD	26
00043A49-2949-494B-A5DD-A5BAE38B19DD	97
00043A49-2949-494B-A5DD-A5BAE38B19DD	87
00045D6B-ED2E-4F2F-8186-CEE074D875D0	30

Result 26

#	Time	Action	Message
1	11:06:09	SELECT tr.id AS transaction_id, jt.product_id FROM fact_transactions tr JOIN JSON_TABLE(CO...	253388 row(s) returned

De los 100 mil registros en fact_transactions. Hay 253388 ids extraídos de la columna producto_ids, ahora vamos a crear la tabla intermedia. Y no menos importante, creamos las relaciones FOREIGN KEY, con la tabla fact_transaction(id) y dim_products(id).

```

319 #Creamos la tabla intermedia para guardar los ids extraidos de la columna product_ids de fact_transaction
320 * CREATE TABLE transaction_products (
321   transaction_id VARCHAR(50),
322   product_id INT,
323   PRIMARY KEY (transaction_id, product_id), # la clave primaria sera compuesta para evitar duplicidad, una misma transaccion puede tener muchos productos asociados
324   FOREIGN KEY (transaction_id) REFERENCES fact_transactions(id), #referenciamos como clave foranea transaction_id con id en fact_transactions
325   FOREIGN KEY (product_id) REFERENCES dim_products(id) # referenciamos product_id con id de dim_products recién creada
326 );

```

#	Time	Action	Message	Duration / s
1	21:30:08	CREATE TABLE transaction_products (transaction_id VARCHAR(50), product_id INT, PRIMARY KEY (transaction_id, product_id), # la clave primaria sera comp. 0 rows) affected		0.109 sec

Usamos la misma query de json table para hacer el insert

```

319 #Creamos la tabla intermedia para guardar los ids extraidos de la columna product_ids de fact_transaction
320 * CREATE TABLE transaction_products (
321   transaction_id VARCHAR(50),
322   product_id INT,
323   PRIMARY KEY (transaction_id, product_id), # la clave primaria sera compuesta para evitar duplicidad, una misma transaccion puede tener muchos productos asociados
324   FOREIGN KEY (transaction_id) REFERENCES fact_transactions(id), #referenciamos como clave foranea transaction_id con id en fact_transactions
325   FOREIGN KEY (product_id) REFERENCES dim_products(id) # referenciamos product_id con id de dim_products recién creada
326 );
327
328 #Ahora realizamos el insert en transaction_products con la funcion anterior que extrae los productos de la columna products_ids en fact_transactions
329 * INSERT INTO transaction_products(
330   transaction_id,
331   product_id
332 )
333 SELECT
334   tr.id AS transaction_id,
335   jt.product_id
336 FROM fact_transactions tr
337 JOIN JSON_TABLE(
338   CONCAT('[', tr.product_ids, ']'), '$[*]' COLUMNS (product_id INT PATH '$')
339   # CONCAT('[', t.product_ids, ']') Añade [ ] con concat porque la funcion JSON lee arrays
340   # '$[*]' el $ sera nuestra columna 't.product_ids' y con [*] recorre todos los elementos del array
341   # COLUMNS extrae cada elemento y lo convierte a columna
342   # (product_id INT PATH '$') lo renombras a product_id, guárdalo como INT de lo extraido con '$' = t.product_ids
343 ) AS jt; # referenciamos la funcion como tabla

```

#	Time	Action	Message	Duration / s
1	21:42:56	INSERT INTO transaction_products(transaction_id, product_id) SELECT tr.id AS transaction_id, jt.product_id FROM fact_transactions tr JOIN JSON_TABLE(CO... 253388 row(s) affected (batch size: 253388) Duration: 0.109 sec		0.109 sec

Verificamos el número de veces que se ha vendido cada producto.

```

737 --
738 #verificamos el numero de veces que se ha vendido cada producto
739 SELECT tp.product_id AS id,p.product_name AS producto,COUNT(*) AS ventas_producto
740 FROM transaction_products AS tp
741 JOIN dim_products AS p ON p.id=tp.product_id
742 GROUP BY product_id,p.product_name;
743
744
745
746

```

id	producto	ventas_producto
1	Direwolf Stannis	2468
2	Tarly Stark	2573
3	duel tourney Lannister	2528
4	warden south duel	2584
5	skywalker ewok	2548
6	rhokkai onlu	2407

Result 29

Output

Action Output

#	Time	Action	Message
1	11:59:08	SELECT tp.product_id AS id,p.produc...	100 row(s) returned

Modelo final:

